Expert Systems With Applications 328 (2026) 132644



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



HalluTrigger: Triggering input-conflicting hallucinations in large language models with semantic-guided metamorphic relations

Chongyang Liu [a], Xiaoning Ren [a,*], Yinxing Xue [b]

[a] Department of Computer Science and Technology, University of Science and Technology of China, 96 Jinzhai Road, Hefei, Anhui, 230026, China
[b] Institute of AI for industries, 168 Tianquan Road, Nanjing, Jiangsu, 211135, China

ARTICLE INFO

Keywords:
Input-conflicting hallucination
Large language model
Metamorphic testing

ABSTRACT

With rapid development of Large Language Models (LLMs), they are increasingly being adopted as knowledge-based intelligent systems to support decision-making across domains such as education, healthcare, business, and government. Alongside these strides, LLMs have also raised concerns about their security and reliability. One such concern is Input-Conflicting Hallucination (ICH), a type of hallucination that conflicts with user input. Since the data annotation in NLP tasks is expensive and labor-intensive, existing ICH attack methods have adopted Metamorphic Testing (MT) to bypass the oracle problem, i.e. check the ICHs without the ground truths. However, these attacks are all black-box methods utilizing a limited set of Metamorphic Relations (MRs) and can be easily defended against by decoder-only LLMs. In response, we propose HalluTrigger, a simple yet effective grey-box method that constructs six semantic-guided MRs to generate attack cases, and propose a diversity-guided MP prioritization method to promote the trigger of ICHs in target LLMs. We evaluate HalluTrigger on four NLP datasets and three popularly used target LLMs. As a result, the designed MRs achieve higher hallucination trigger rates than existing state-of-the-art baselines, and the diversity-guided MP prioritization can efficiently trigger ICHs with less time.

1. Introduction

With dramatic evolution in recent years, Large Language Models (LLMs) such as BART (Lewis et al., 2019), GPT4 (OpenAI, 2023), Claude (Anthropic, 2023) and LLaMA 3 (LlamA, 2024) have made unprecedented progress in natural language understanding (Hendrycks et al., 2020) and generation (Zhang et al., 2024b), leading popular applications in resolving various downstream tasks. As LLMs gain prominence in academia, industry, and daily use, their security and reliability have become an increasingly important topic. Input-conflicting hallucination (ICH), a type of hallucination that conflicts with source input provided by users, has posed a significant challenge due to the free-form and lengthy nature of LLMs generations (Huang et al., 2023; Zhang et al., 2023). In scenarios such as natural language inference and question answering, correctness is primarily determined by the input semantics and task specifications, rather than by external world knowledge or long conversational context as in other types of hallucinations. This distinction makes it essential to systematically test and analyze ICH behaviors in LLMs.

In ICH attacks for LLMs, a key step is verifying the correctness of LLMs' outputs, also known as the oracle problem in Software Engineering (Chen et al., 2020). This typically relies on manual inspection and is complex and time-consuming to annotate the generations of LLMs precisely (Dale et al., 2022; Dziri et al., 2022; Guerreiro et al., 2022; Zhong et al., 2021). Metamorphic Testing (MT) is a solution to bypass the oracle problem. Given an existing case (called the source case) and the LLM under test, MT generates a new case (called the follow-up case) based on a Metamorphic Relation (MR) and constructs a Metamorphic test case Pair (MP). Note that a MP is just a special case of a metamorphic group. The MR is the core component of MT, it defines how to generate a follow-up case from the source case and the expected relation between the outputs of the two cases. After executing the MP on target LLM, MT inspects the relation between the outputs and MR. Once an ICH is triggered if the two outputs violate the MR.

Existing studies have proposed a few MT-based methods to reveal the violations in LLMs, which are ICHs through our analysis. Chen et al. (2021) propose QAAskeR to generate new questions and answers based on the synthesized pseudo facts derived from original questions and generated answers. Shen et al. (2022) proposed QAQA to construct MPs by inserting redundant sentences searched from the training set into source cases. Liu et al. (2022) propose QATest to generate follow-up cases by grammatical component-based, sentence structural-based, and

* Corresponding author.
E-mail addresses: lcyyy@mail.ustc.edu.cn (C. Liu), hnurxn@mail.ustc.edu.cn (X. Ren), yxxue@iaii.ac.cn (Y. Xue).

https://doi.org/10.1016/j.eswa.2026.132644
Received 30 August 2025; Received in revised form 5 January 2026; Accepted 24 April 2026
Available online 19 May 2026
0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

adversarial perturbation-based transformations. However, these attacks: (i) solely consider the syntax information and generate low-quality follow-up cases, (ii) mainly aim to attack two types of LLMs, i.e. UnifiedQA (Khashabi et al., 2020) and ALBERT (Lan et al., 2020), and have poor attack efficacy on decoder-only LLMs, and (iii) are black-box without utilizing the hidden outputs to identify ICH-sensitive MPs and facilitate the attack process.

To address the aforementioned issues, we propose a grey-box attacking method named HalluTrigger to effectively and efficiently trigger the ICHs in LLMs. HalluTrigger consists of two components: a multi-level MP construction module and a diversity-based MP prioritization module. In the first module, we design 4 MRs at the character level, one MR at the word level, and one MR at the semantic level. Each MR generates a set of source/follow-up case pairs, not just a single pair, and all of them can be applied to general NLP datasets and LLMs to reveal potential ICHs. Besides, we consider the syntax and semantic information to ensure the quality of generated cases. In the second module, inspired by the test case prioritization method in traditional software and DNN testing (Cao et al., 2013; Xie et al., 2022), we involve two diversity metrics, i.e. Jensen-Shannon divergence (JS) and Hallengle Distance (HD), to perform MP prioritization and trigger ICHs efficiently with the prioritized MP sequences. Note that the diversity of each MP is calculated based on the hidden outputs of the decoder blocks in target LLMs. To measure the performances of the diversity-based MP prioritization, we propose the Average Percentage of Hallucination Triggered (APHT) and normalized APHT (NAPHT). In summary, our primary contributions are as follows:

- Effective MRs for triggering ICHs in LLMs. We propose six semantic-guided MRs that can be applied to different types of LLMs and NLP tasks.
- Efficient grey-box MP prioritization method. We make the first attempt to identify the ICH-sensitive MPs with LLMs' hidden outputs and prioritize them with two diversity metrics. We propose a new performance metric to measure the performance of our prioritization method.
- Extensive evaluation of HalluTrigger. We conduct the experiments on several popularly used NLP tasks and LLMs. The results indicate that our method can both effectively and efficiently trigger ICHs in target LLMs.
- Security enhancement through adversarial fine-tuning. We select the ICH-sensitive MPs from the training set based on diversity metrics and perform adversarial fine-tuning to enhance the security of target LLMs.

2. Background and related work

2.1. Hallucinations in LLMs and their evaluation

Prior studies have shown that LLMs are facing severe security and reliability challenges, such as backdoor attacks and hallucinations. In the context of backdoor attacks, ProAttack (Zhao et al., 2025) leverages prompt-based clean-label poisoning to insert backdoors during training without external triggers. Hu et al. (2025) use LLM-generated syntactic paraphrases as implicit triggers, achieving robust and highly stealthy attacks. BadMerging (Zhang et al., 2024a) demonstrates that merging a single backdoored contributor model can compromise the merged model.

In contrast, hallucinations, particularly input-conflicting hallucination (ICH), in which the output contradicts user-provided input despite semantic equivalence, pose a distinct challenge and are the focus of this work. Referring to Huang et al. (2023), Zhang et al. (2023), hallucinations in LLMs can be broadly categorized into the following three types .

Context-Conflicting Hallucination (CCH) arises when LLMs lose track of the generated contents or cannot maintain consistency throughout the conversation. As a result, the LLMs may have self-contradictions in generating lengthy or multi-turn contents. Fig. 1 (c) shows an instance of conflict response. Fact-conflicting hallucination (FCH) occurs when LLMs generate information or texts that contradict the constructed world knowledge. The reason is complicated due to various factors introduced at different stages of the LLM life cycle. In Fig. 1 (d), LLM outputs a response that is inconsistent with reality.

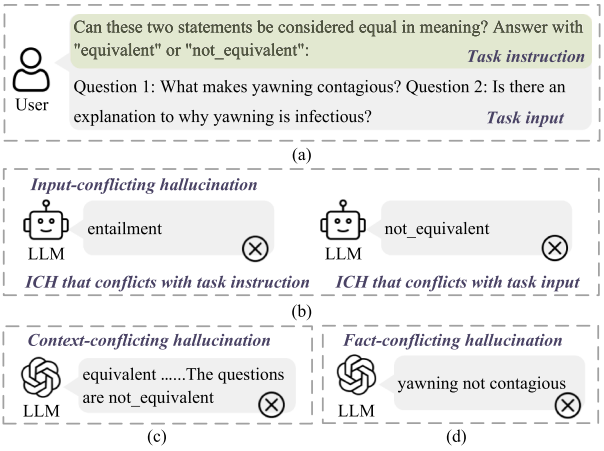


Fig. 1. An example of the format of user input and input-conflicting hallucinations occurred in LLM responses.

ICH occurs when the content generated by LLMs deviates from the input provided by the users. As shown in Fig. 1 (b), it can happen in two ways: the LLM's responses contradict the task instructions or the LLM's responses contradict the task input. This means the LLMs misunderstand the users' intents or representations of the context. Although ICH is relatively easy for users to identify according to the contexts, evaluating it within LLMs is particularly challenging since LLM generates content in a free and lengthy format. For these reasons, we focus on the ICH in decoder-decoder and decoder-only LLMs in this work.

Recent hallucination evaluation methods in NLP can be categorized as follows: (i) human evaluation (Lin et al., 2021; Min et al., 2023) mainly focuses on conducting human annotations or guidelines to ensure reliable evaluation, (ii) model-based automatic evaluation (Zha et al., 2023) generally trains a proxy LLM to evaluate whether the answer of the subject model is correct, and (iii) rule-based automatic evaluation (Bang et al., 2023; Lee et al., 2022; Yu et al., 2023) uses classic classification metrics such as accuracy and F1 in classification tasks. These methods all rely on the analysis of the oracle. For these reasons, we adopt MT to bypass the oracle problem (i.e., verify whether the outputs are ICHs without ground truths) and promote revealing ICHs in LLMs.

2.2. Metamorphic testing

MT is quite a popular technique due to its ability to effectively bypass the oracle problem (Chen et al., 2020) and uncover the defects in target software or DNNs. MT can be used both to generate test cases and verify the correctness of results. A central component of MT is a set of Metamorphic Relations (MRs), which define the expected relation between the target model, its inputs, and the outputs. Given a set of source cases, MT uses the defined MRs to generate new test cases as follow-up test cases and construct Metamorphic test case Pairs (MPs). Instead of verifying the correctness of the outputs for the source and follow-up cases in each MP, MT checks whether the outputs adhere to the corresponding MR. If an MR is violated, at least one output is hallucination.

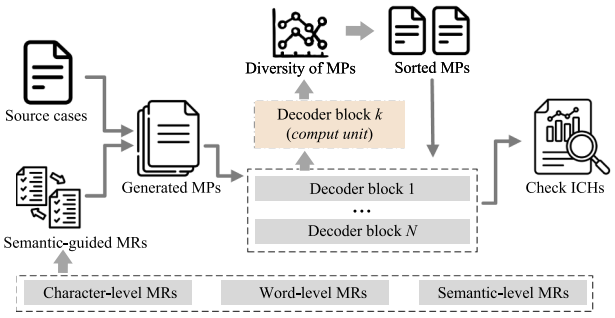


Fig. 2. Overview of HalluTrigger framework with semantic-guided MP construction and diversity-based prioritization.

2.3. Metamorphic testing for hallucination detection

Existing MT-based studies mainly focus on detecting ICHs in UnifiedQA and ALBERT, with some extensions toward broader hallucination types such as FCH. For ICH detection, QAAskeR (Chen et al., 2021) constructs MRs based on the synthesized pseudo facts derived from questions and answers, which may introduce false positives due to weak semantic constraints. QAQA (Shen et al., 2022) emphasizes semantic faithfulness by preserving answers under equivalent and inferential relations, but is limited to QA-style datasets with explicit contexts. QATest (Liu et al., 2022) employs grammar- and structure-level perturbations to enhance attack strength, yet these transformations may overly distort semantics, blurring the boundary between hallucination and legitimate semantic shifts. For broader hallucination detection, HalluVault (Li et al., 2024) designs logic-rule-based MRs to detect FCHs, focusing on external knowledge violations rather than ICHs.

From a different perspective, some studies employ LLMs not as the system under test, but as auxiliary test generators. FuzzCoder (Yang et al., 2025) employs code LLMs to guide fuzz testing by generating semantically valid and diverse program mutations for industrial IoT systems.

Positioned within MT-based ICH detection, HalluTrigger aims to effectively and efficiently uncover ICHs in widely used open-source LLMs by integrating syntactic and semantic constraints.

3. HalluTrigger

3.1. Overview

Fig. 2 shows the overview of HalluTrigger, which can be divided into two modules: a multi-level MP construction module and a diversity-guided MP prioritization module. In the first module, we propose semantic-guided MRs at different levels use them to generate follow-up cases and construct MPs. Specifically, we propose four MRs at the character level, one MR at the word level, and one MR at the semantic level. In the second module, we select a decoder block from each target LLM as the *compute unit*, adopt two diversity metrics to calculate the diversity of each MP with the outputs of the compute unit, and sort all MPs in descending order. Refer to Cao et al. (2013), MPs with high diversity are considered ICH-sensitive and are prioritized input into target LLMs to check the results.

3.2. Problem formulation

3.2.1. Oracle and ICH

Generally, the test oracles for different types of tasks vary. The downstream tasks of LLMs are primarily divided into classification tasks and generation tasks. Given the human oracle $\mathcal{O}$, the source case $\mathcal{X}$, and a target LLM $\mathcal{LM}$, we design an MR $\mathcal{M}$ and generate a follow-up case $\mathcal{X}'$. The set of generated follow-up cases is $\mathcal{M}(\mathcal{X})$.

In classification tasks such as duplicate sentence detection (DSD), the output of LLMs is the concrete label (e.g. “equivalent” or “not_equivalent”) of the corresponding input. For $\forall \mathcal{X}' \in \mathcal{M}(\mathcal{X})$, the oracle of target LLM $\mathcal{LM}$ can be defined as follows:

$$\mathcal{O}(\mathcal{X}') = \mathcal{O}(\mathcal{X}) \tag{1}$$

Then $\mathcal{X}'$ is an ICH if it satisfies the requirements:

$$\mathcal{LM}(\mathcal{X}') \neq \mathcal{LM}(\mathcal{X}), \quad \text{s.t. } \mathcal{LM}(\mathcal{X}) = \mathcal{O}(\mathcal{X}) \tag{2}$$

In generative tasks such as reading comprehension (RC), due to the lack of a standardized format in LLM outputs, we refer to previous study (Shen et al., 2022) and determine their accuracy by comparing the semantic similarity Sim between the output $\mathcal{LM}(\mathcal{X})$ and the reference answer $\mathcal{A}$.

$$Sim(\mathcal{LM}(\mathcal{X}), \mathcal{A}) = \frac{embed(\mathcal{LM}(\mathcal{X}))^T \cdot embed(\mathcal{A})}{\|embed(\mathcal{LM}(\mathcal{X}))\| \|embed(\mathcal{A})\|} \tag{3}$$

where $embed(\cdot)$ is the embedding function integrated in Phrase-BERT (Wang et al., 2021) and $Sim(\cdot)$ is the cosine similarity function. For $\forall \mathcal{X}' \in \mathcal{M}(\mathcal{X})$, the oracle of target LLM LM can be defined as follows:

$$\mathcal{O}(\mathcal{X}') = \mathcal{O}(\mathcal{X}), \quad \text{s.t. } Sim(\mathcal{LM}(\mathcal{X}), \mathcal{LM}(\mathcal{X}')) \geq \gamma \tag{4}$$

where γ is the similarity threshold. In this scenario, $\mathcal{X}'$ is an ICH if it satisfies the requirements:

$$Sim(\mathcal{LM}(\mathcal{X}), \mathcal{LM}(\mathcal{X}')) < \gamma, \quad \text{s.t. } \mathcal{LM}(\mathcal{X}) = \mathcal{O}(\mathcal{X}) \tag{5}$$

3.2.2. Attack goal

Given a set of source cases as input S, each case that can be correctly recognized by target LLM $\mathcal{LM}$ constitutes a set $S' \subset S$. The attack goal contains two aspects: (i) *maximization of the hallucination trigger rate (HTR)*, where HTR is the proportion of follow-up cases induce outputs violating semantic constraints and a higher HTR indicate a greater effectiveness. Let $S' = \{\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_{|S'|}\}$ where $\mathcal{X}_i$ is the ith source case in S', $|S'|$ is the size of S', and $\mathcal{X}'_i$ is the corresponding follow-up case generated by MR $\mathcal{M}$. In classification tasks, the HTR of $\mathcal{M}$ on S can be calculated as follows:

$$\text{HTR}_{\mathcal{M}} = \frac{\sum_{i=1}^{S'} \left[\mathcal{LM}(\mathcal{X}'_i) \neq \mathcal{LM}(\mathcal{X}_i)\right]}{|S'|} \tag{6}$$

In generation tasks, the HTR can be calculated as follows:

$$\text{HTR}_{\mathcal{M}} = \frac{\sum_{i=1}^{S'} \left[Sim(\mathcal{LM}(\mathcal{X}), \mathcal{LM}(\mathcal{X}')) < \gamma\right]}{|S'|} \tag{7}$$

Then we formulate the problem in this aspect as follows:

$$\mathcal{M} \leftarrow \arg\max_{\mathcal{M}} \text{HTR}_{\mathcal{M}} \tag{8}$$

We aim to construct an MR $\mathcal{M}$ to maximize $\text{HTR}_{\mathcal{M}}$, and (ii) *maximization of Average Percentage of Hallucination Triggered (APHT)*. Given a generated pile of MPs $\Gamma = \{\mathcal{MP}_1, \mathcal{MP}_2, \ldots, \mathcal{MP}_{|\Gamma|}\}$ and the prioritized sequence of MPs $\Gamma' = \left\langle \mathcal{MP}'_1, \mathcal{MP}'_2, \ldots, \mathcal{MP}'_{|\Gamma|} \right\rangle$, where $\mathcal{MP}_i$ is the ith MP and $|\Gamma|$ is the total number of MPs in Γ. Inspired by test case ranking metrics proposed in traditional software and DNN testing (Rothermel et al., 1999; Xie et al., 2022), we design a new rank-based metric (i.e. APHT) to evaluate the performance of our prioritization method:

$$\text{APHT}(\Gamma') = 1 - \frac{\sum_{i=1}^{m} \mathcal{RH}_i}{|\Gamma| \cdot m} + \frac{1}{2 \cdot |\Gamma|} \tag{9}$$

where m denotes the numbers of ICH-sensitive MPs in Γ', and $\mathcal{RH}_i$ is the rank of the ith ICH-sensitive MP in Γ'. A higher APHT indicates that the ICH-sensitive cases are well identified and ranked. Furthermore, considering the reachable upper and lower bounds of APHT, we scale it into the normalized APHT (NAPHT) which is calculated as follows:

$$\text{NAPHT}(\Gamma') = \frac{\text{APHT}(\Gamma') - \text{APHT}_{min}}{\text{APHT}_{max} - \text{APHT}_{min}} \tag{10}$$

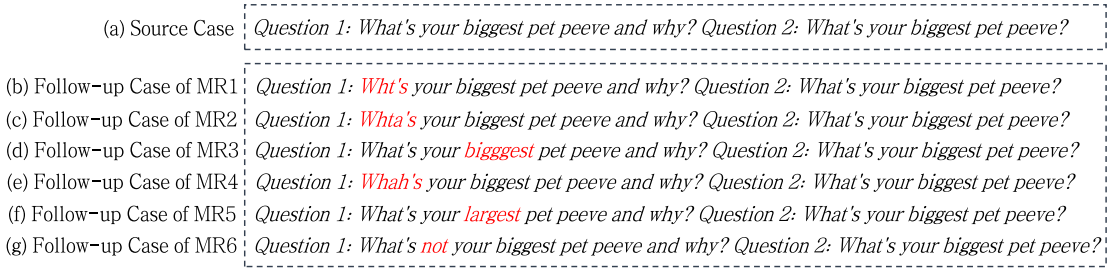


Fig. 3. An example of source case and corresponding follow-up cases under six MRs.

where $\text{APHT}_{\max}$ refers to the APHT that prioritizes the ICH-sensitive MPs from 1 to m, and $\text{APHT}_{\min}$ refers to the APHT that prioritizes the ICH-sensitive MPs from $|\Gamma| - m + 1$ to $|\Gamma|$. Then we formulate the second problem as follows:

$$D \leftarrow \arg\max_{D} \text{NAPHT}(\Gamma') \tag{11}$$

We aim to utilize a diversity metric D to improve the effectiveness of MP prioritization, i.e. maximize $\text{NAPHT}(\Gamma')$.

3.3. Attack methodology

3.3.1. Multi-level MP construction

In our method, it is necessary to construct a series of MRs to support the process of MT and then trigger ICHs in target LLMs. We comprehensively analyze the linguistic structure of LLM's input and construct MRs at the character, the word, and the semantic level to attack target LLMs effectively. Refering to previous research (Alzantot et al., 2018; Rawlinson, 2007), our constructed MRs either introduce slight perturbations to texts or make logically consistent changes.

At the character level and word level, the source case and its follow-up are semantically consistent, such that their corresponding output labels are identical or the cosine similarity between their output texts is greater than or equal to γ. There are two main steps in these two scenarios: (i) find the most vulnerable words in the source cases, and (ii) perturb the vulnerable words, to generate MPs. Given a source case $\mathcal{X} = \{x_1, x_2, \ldots, x_{|\mathcal{X}|}\}$ and a BERT model $\mathcal{F}$, where x_i is the ith word and $|\mathcal{X}|$ is the number of all words in $\mathcal{X}$. The corresponding output is $\mathcal{F}(\mathcal{X}) = \{o_1, o_2, \ldots, o_{|\mathcal{X}|}\}$ where o_t is the output logit for time step t. To find the most vulnerable word in $\mathcal{X}$, we replace x_i with `[MASK]` such that $\mathcal{X}_{\setminus x_i} = \{x_1, \ldots, x_{i-1}, \texttt{[MASK]}, x_{i+1}, \ldots, x_{|\mathcal{X}|}\}$. The output of $\mathcal{X}_{\setminus x_i}$ is $\mathcal{F}(\mathcal{X}_{\setminus x_i}) = \{o'_1, o'_2, \ldots, o'_{|\mathcal{X}|}\}$ where o'_t is the new output logit for time step t. The influence score for x_i is as follow:

$$\mathcal{I}_{x_i} = \left| \sum_{t=1}^{|\mathcal{X}|} o_t - \sum_{t=1}^{|\mathcal{X}'|} o'_t \right| \tag{12}$$

We finally choose the x_i with the highest influence score.

At the character level, despite the applied modifications, the semantic content of the text is largely preserved and remains inferable to human readers, thereby ensuring semantic consistency between the source and follow-up input cases. Specifically, as shown in Fig. 3 (a), given a source case ‘‘`Question 1: What's your biggest pet peeve and why? Question 2: What's your biggest pet peeve?`’’ with label "entailment", we randomly alter characters within words of it and define four MRs:

- **MR1: RandomCharacterDeletion (RCDel).** RCDel generates follow-up cases by randomly deleting one character from the word to the source case, excluding the first and last characters. Fig. 3 (b) illustrates an example in which the character "a" in "what's" is deleted. This modification triggers an ICH in the target LLM if the model outputs "not entailment."
- **MR2: NeighboringCharacterSwap (NCSwa).** NCSwa generates follow-up cases by randomly swapping the neighboring character in the word at once to the source case. Fig. 3 (c) illustrates an example in which the character "a" and "t" in "what's" is swapped. This modification triggers an ICH in the target LLM if the model outputs "not entailment."
- **MR3: RandomCharacterInsertion (RCIns).** RCIns generates follow-up cases by randomly inserting a character between the first and the last characters in the word to the source case. Fig. 3 (d) illustrates an example in which the character "g" is inserted in "biggest". This modification triggers an ICH in the target LLM if the model outputs "not entailment."
- **MR4: RandomCharacterSubstitution (RCSub).** RCSub generates follow-up cases by randomly substituting a character in the word to the source case. Fig. 3 (e) illustrates an example in which the character "t" in "what's" is substituted by "h". This modification triggers an ICH in the target LLM if the model outputs "not entailment."

At the word level, we mask the identified vulnerable words and employ a Mask Language Model (i.e., BERT-MLM) to generate N candidate substitutions. Given a source case $\mathcal{X}$ and its corresponding follow-up case $\mathcal{X}'$, let x_i and x'_i denote the word before and after perturbation, respectively. To ensure both naturalness and semantic consistency, we follow AEON (Huang et al., 2022) and impose the following constraints on the quality of $\mathcal{X}'$:

$$Nat(\mathcal{X}, \mathcal{X}') = \left| \sqrt{\Pi_{i=1}^{|\mathcal{X}|} P(x_i | \mathcal{X}_{\setminus x_i})} - \sqrt{\Pi_{i=1}^{|\mathcal{X}'|} P(x'_i | \mathcal{X}'_{\setminus x'_i})} \right| \tag{13}$$

$$Score_{\mathcal{X}'} = -\lambda_1 Nat(\mathcal{X}, \mathcal{X}') + \lambda_2 Sim'(\mathcal{X}, \mathcal{X}') \tag{14}$$

where λ_1 and λ_2 denote weighting coefficient, and $Sim'(\cdot)$ represents the similarity function proposed in *SimCSE* (Gao et al., 2021). A candidate x' is accepted with score greater than or equal to φ (i.e., $Score_{x'} \geq \varphi$).

Based on the above semantic consistency guarantees, we define one MR for this scenario:

- **MR5: SimilarWordSubstitution (SWSub).** SWSub generates follow-up cases by replacing the original word in the source case with a word selected from the top-k candidates. Fig. 3 (f) illustrates an example in which the word "biggest" is replaced with "largest". This modification triggers an ICH in the target LLM if the model outputs "not entailment."

At the semantic level, unlike the character- and word-level settings that aim to preserve the original semantics, we introduce precise and controllable semantic negations to alter the meaning of the input and further explore ICH behaviors in target LLMs. Specifically, We leverage the NLTK toolkit (Bird et al., 2001) to construct syntactic parse trees and apply linguistically grounded transformations (e.g., negating copular or auxiliary verbs), ensuring grammaticality and naturalness of the generated follow-up cases.

Since semantic negation is not semantic-preserving, the outputs of a source case $\mathcal{X}$ and its corresponding follow-up case $\mathcal{X}'$ are expected to follow task-specific semantic relations. Given a target model $\mathcal{LM}$, these expected relations are defined as:

- In MNLI, if the output of $\mathcal{X}$ is "neutral", then $\mathcal{LM}(\mathcal{X}') = \mathcal{LM}(\mathcal{X})$; otherwise $\mathcal{LM}(\mathcal{X}') \neq \mathcal{LM}(\mathcal{X})$.
- In QQP, $\mathcal{LM}(\mathcal{X}') = \mathcal{LM}(\mathcal{X})$.
- In SQuAD2, if the output of $\mathcal{X}$ is "unanswerable", then the output of $\mathcal{X}'$ is "unanswerable"; otherwise $\mathcal{LM}(\mathcal{X}') \neq \mathcal{LM}(\mathcal{X})$.
- In NarrativeQA, $Sim(\mathcal{LM}(\mathcal{X}), \mathcal{LM}(\mathcal{X}')) < \gamma$.

Any violation of the above expected relations indicates that the target model exhibits ICH behavior. Based on these task-specific semantic negation rules, we define one MR as follows:

- **MR6: Semantic Negation (SNeg)**. Sneg generates follow-up cases by applying semantic negation operations (e.g., negating copular or auxiliary verbs) to the source case. Fig. 3 (g) illustrates an example in which the "what's" is negated to "what' not". This modification triggers an ICH in the target LLM if the model outputs "entailment."

3.3.2. Diversity-guided MP prioritization

In traditional software testing and DNN testing, prior studies have shown that execution diversity of metamorphic paths is positively correlated with the likelihood of triggering violations (Cao et al., 2013; Chen et al., 2004; Xie et al., 2022). Motivated by this well-established hypothesis, we prioritize MPs for LLMs under the guidance of execution diversity. Given a LLM $\mathcal{LM} = \{\mathcal{DB}_1, \mathcal{DB}_2, \ldots, \mathcal{DB}_{|\mathcal{LM}|}\}$, the $\mathcal{MP}_i = (\mathcal{X}_i, \mathcal{X}_i')$, and the generated pile of MPs Γ, where $\mathcal{DB}_k$ is the kth decoder block and $|\mathcal{LM}|$ is the total number of decoder blocks in $\mathcal{LM}$.

The first key component of this module is the diversity metrics. In HalluTrigger, we introduce two accurate and cheap implementation diversity metrics that work effectively on DNNs (Xie et al., 2022), i.e. *Jensen-Shannon divergence (JS)* and *Hellinger distance (HD)*. JS measures the difference between two distributions based on *Kullback-Leibler divergence (KL)*, it is symmetric and bounded. To calculate the JS-based diversity of $\mathcal{MP}_i$ and $\mathcal{DB}_k$, we first calculate its KL-based diversity as follows:

$$\mathcal{D}_{KL}(\mathcal{X}_i||\mathcal{X}_i') = \sum_{n_j^k \in \mathcal{L}_k} \mathcal{DB}_k(\mathcal{X}_i, n_j^k) \ln \frac{\mathcal{DB}_k(\mathcal{X}_i, n_j^k)}{\mathcal{DB}_k(\mathcal{X}_i', n_j^k)} \tag{15}$$

where $\mathcal{DB}_k(\mathcal{X}_i, n_j^k)$ and $\mathcal{DB}_k(\mathcal{X}_i', n_j^k)$ denotes the outputs of the jth neuron n_j^k in last full connected layer $\mathcal{L}_k$ of $\mathcal{DB}_k$ for $\mathcal{X}_i$ and $\mathcal{X}_i'$, respectively. Then the corresponding JS-based diversity can be calculated as follows:

$$\mathcal{D}_{JS} = \frac{1}{2}(\mathcal{D}_{KL}(\mathcal{DB}_k(\mathcal{X}_i)||M) + \mathcal{D}_{KL}(\mathcal{DB}_k(\mathcal{X}_i')||M)) \tag{16}$$

where $M = \frac{1}{2}(\mathcal{DB}_k(\mathcal{X}_i) + \mathcal{DB}_k(\mathcal{X}_i'))$, and $\mathcal{DB}_k(\mathcal{X}_i)$ and $\mathcal{DB}_k(\mathcal{X}_i')$ is the final output of $\mathcal{DB}_k$ for $\mathcal{X}_i$ and $\mathcal{X}_i'$, respectively. The HD-based diversity calculates the divergence between two distributions using the Hellinger integral. This type of diversity metric is calculated as follows:

$$\mathcal{D}_{HD} = \sqrt{\frac{\sum_{n_j^k \in \mathcal{L}_k} \left(\sqrt{\mathcal{DB}_k(\mathcal{X}_i, n_j^k)} - \sqrt{\mathcal{DB}_k(\mathcal{X}_i', n_j^k)}\right)^2}{2}} \tag{17}$$

The second key component is the MP prioritization algorithm. We choose the kth decoder block $\mathcal{DB}_k$ as the *compute unit* and compute the diversity of each MP with the aforementioned metrics based on the output of $\mathcal{DB}_k$. Specifically, for each $\mathcal{MP}$ in the generated pile of MPs Γ, we execute the source case $\mathcal{X}$ and the follow-up case $\mathcal{X}'$ in target LLM $\mathcal{LM}$, extract the outputs of the compute unit $\mathcal{DB}_k$ to calculate the diversities. After that, all the MPs are sorted in descending order. The details are shown in Algorithm 1. Generally, MPs with high execution diversity are regarded as ICH-sensitive (Cao et al., 2013; Xie et al., 2022). Their outputs will therefore be prioritized for inspection to identify potential ICHs efficiently.

Algorithm 1 Diversity-guided MP prioritization.

Input: a pile of original MPs Γ, compute unit $\mathcal{DB}_k$.
Output: a sequence of prioritized MPs Γ'.
Initilize: $\Gamma' \leftarrow \emptyset$.

for $\mathcal{MP}$ in Γ **do**
  $\mathcal{X}, \mathcal{X}' \leftarrow \mathcal{MP}$
  $output_{\mathcal{X}} = \mathcal{DB}_k(\mathcal{X})$
  $output_{\mathcal{X}'} = \mathcal{DB}_k(\mathcal{X}')$
  // Compute the execution diversity of $\mathcal{MP}$
  $\mathcal{D}_{\mathcal{MP}} = Diversity(output_{\mathcal{X}}, output_{\mathcal{X}'})$
  $\Gamma' \leftarrow \Gamma' \cup (\mathcal{MP}, \mathcal{D}_{\mathcal{MP}})$
end for
// Prioritize all the MPs based on computed diversities
$\Gamma' = Prioritization(\Gamma')$

4. Experiments

We study the following research questions:

- **RQ1:** How effective are the proposed semantic-guided MRs in triggering ICHs?
- **RQ2:** Does the proposed MP prioritization algorithm improve the efficiency of triggering ICHs?
- **RQ3:** Can adversarial fine-tuning with ICH-sensitive MPs enhance the security of LLMs?
- **RQ4:** What impact does the selection of compute unit have in the MP prioritization method?

4.1. Downstream tasks and datasets

To comprehensively evaluate HalluTrigger, we conduct our experiments on both classification and generation tasks. Referring to existing studies (Chen et al., 2021; Duan et al., 2024; McCoy et al., 2019; Shen et al., 2022; Yuan et al., 2024), we perform our experiments on the following NLP tasks and datasets: (i) three-category natural language inference (NLI) task based on MNLI (Williams et al., 2017), (ii) binary classification duplicate sentence detection (DSD) task based on QQP (Wang et al., 2018), and (iii) generative reading comprehension (RC) task based on SQuAD2 (Rajpurkar et al., 2018) and NarrativeQA (Kočiskỳ et al., 2018). The details are shown as follows:

- **MNLI** dataset is a large-scale dataset composed of sentence pairs with textual entailment annotations, where each sentence pair contains a *Premise* and a *Hypothesis*. It is designed to evaluate a model's ability to understand and infer the relationship between the *Premise* and *Hypothesis*. Based on MNLI, NLI is a three-category task, containing three labels: "entailment", "neutral" and "contradiction".
- **QQP** dataset is a collection of question pairs from the community question-answering website Quora, annotated with whether the pairs are semantically equivalent. Based on QQP, DSD is a binary classification task, with labels of "equivalent" and "not_equivalent".
- **SQuAD2** is an extractive reading comprehension dataset in which the contexts, questions, and answers are collected from Wikipedia articles by crowdworkers. Each question's answer in the dataset is a segment or a certain span from the context. SQuAD2 also includes over 50,000 adversarially unanswerable questions, with corresponding answers marked as "unanswerable".
- **NarrativeQA** is an abstractive reading comprehension dataset that consists of stories in the form of books and movie scripts. It contains questions that require deep understanding and reasoning over the entire narrative to answer correctly. The answer to each question is not limited to specific spans of text within the story.

Table 1 shows the statistics of these four datasets. In particular, the testing set in MNLI contains the matched and mismatched testing sets.

4.2. Target models

In this work, we aim to attack the generative encoder-decoder and decoder-only LLMs and consider BART, LLaMA, and Vicuna as the target

Table 1
The format and statistics of the downstream datasets.

Dataset	Format	Train/Test
MNLI	Etailment/Neutral/Contradiction	392,702/19,647
QQP	Equivalent/Not_equivalent	363,870/40,431
SQuAD2	Extractive	130,319/11,873
NarrativeQA	Abstractive	32,747/3,461

Table 2
Performance of target LLMs fine-tuned on downstream datasets.

Model	MNLI	QQP	SQuAD2	NqrrativeQA
Vicuna-13B	85.41%	84.90%	81.60%	80.60%
LLaMA-3-8B	83.87%	83.10%	78.70%	81.40%
BART-large	82.06%	82.50%	76.60%	–

LLMs. Specifically, we chose BART-large, the latest development Meta-LLaMA-3-8B, and the version of Vicuna-13b-1.5-16k, where Vicuna-13b-1.5-16k is fine-tuned on LLaMA 2. We select these three LLMs for the following reasons: (1) the target LLMs should be open-source since we need to extract the hidden outputs from them; (2) the target LLMs should be scalable and deployable within our limited computational resources; (3) the target LLMs should achieve state-of-the-art performances on general NLP tasks. As far as we know, these LLMs satisfy the above three requirements and the latter two LLMs achieve over 90% capability of Bard/ChatGPT (Chiang et al., 2023). Hence, we consider them the best-fit subject LLMs to deliver representative results and insights. To facilitate the evaluation of model performance, we refer to PromptBench (Zhu et al., 2023) and adopt different task instructions for tasks to guide the model in generating content in a fixed format (shown in Table 3). In our experiments, all the target models are fine-tuned with LoRa (Hu et al., 2021). Through an empirical study, we fine-tune the target models with different sizes of datasets and find that: (i) BART-large achieves the best performances with 1500 cases, and (ii) Vicuna-13B and LLaMA-3-8B achieve the best performances with 600 cases. Table 2 depicts the accuracy of the fine-tuned target models. Note that the accuracy of each fine-tuned BART-large model on NarrativeQA dataset is less than 35%, and we do no conduct any experiment in this scenario.

4.3. Attack baselines

We compare HalluTrigger with two SOTA MT-based attack methods, and use the most effective MR in each attack method as the baselines for a fair comparison: (i)*QAQA* (Shen et al., 2022): Inserts another input from the training set as redundant sentence into a given source case to generate a follow-up case, and (ii) *QATest* (Liu et al., 2022): Perturbs the source case with typos to generate follow-up case.

4.4. Evaluation metrics

We use the hallucination trigger rate (HTR) and the true positive rate (TPR) to evaluate the effectiveness of the proposed MRs, and use NAPHT to assess the efficiency of the MP prioritization method.

4.5. Implementation details

We conduct the experiments with the Ubuntu 18.04 system with 32-core 2.1GHz Xeon CPU, 196GB RAM and 4 NVIDIA GeForce RTX 3090 24G GPUs. We generate 5 and 1 candidates for each source case at the character and word level, respectively. At the word level, we set the N and k as 30 and 5, respectively. Through manual inspection and analysis of the experiment results, we set the quality φ, weight λ_1 and λ_2 as 0.2, 0.5, and 0.5, respectively. Following QAQA (Shen et al., 2022), we conduct a grid search on manually labeled output pairs and select the threshold that maximizes the performance of the oracle assessment technique in terms of F1 score, resulting in $\gamma = 0.76$. MRs in HalluTrigger are applied to *Premise* or *Question 1* in classification tasks, and to the questions in generation tasks.

4.6. RQ1: effectiveness of semantic-guided MRs

4.6.1. Attack effectiveness

For each dataset, we used the whole dataset to calculate the HTR, and randomly sampled 100 ICHs for manual review to calculate the TPR. Tables 4 and 5 show the results of HTR and TPR, respectively. In most scenarios, the proposed MRs have higher HTR than the SOTA baselines, especially the sentence-level and semantic-level MRs. Besides, the TPR of HalluTrigger on each dataset and target model is no less than 97%. These indicate that HalluTrigger can effectively trigger ICHs in target LLMs the triggered ICHs are almost true positives.

4.6.2. Quality of MPs

We visualize the outputs of the last decoder block of target models with t-SNE technique (Van der Maaten & Hinton, 2008), and analyze the distribution between the source cases and follow-up cases. The rationale is that the reported bugs are meaningful for the system if the generated cases are distributed consistently with the original cases (Berend et al., 2020). Fig. 4 depicts the distribution of MPs on a subset of MNLI in Vicuna-13B, LLaMA-3-8B, and BART-large. We can observe that the distribution within all the MPs is consistent, indicating the high quality of generated follow-up cases.

4.7. RQ2: effectiveness of MP prioritization

4.7.1. Evaluation with NAPHT

In this study, we set the implementation of no prioritization as the baseline (None), and select the middle decoder block in target models as the compute unit. In other settings, our prioritization method is also effective (see more details in Section 4.9). Figs. 5–7 shows the results on Vicuna-13B, LLaMA-3-8B and BART-large, respectively. We find that applying our prioritization method with any diversity metric consistently increases NAPHT compared to no prioritization, demonstrating its effectiveness. Notably, HD outperforms JS in guiding MP prioritization in a model-dependent manner, particularly for larger instruction-tuned LLMs.

4.7.2. Analysis on efficiency

We also execute the unsorted and sorted MP sequences, and record the number of triggered ICHs over time (min) to further analyze the efficiency of the prioritization method. Fig. 8 depicts the result on SQuAD2 and Vicuna-13B. Intuitively, compared with the unsorted MP sequences, The sorted MP sequence can trigger more ICHs within the same time and requires less time to trigger the same number of ICHs.

4.8. RQ3: adversarial fine-tuning

To answer this RQ, we aim to study whether our proposed MRs and the adopted diversity metrics can construct a new dataset that helps mitigate the phenomenon of ICH in LLMs. Exclude the data used to fine-tune target models in Section 4.1, we use the filtered training set as the seed pool. Considering the training set is enormous and the model achieves a comparable performance with 600 cases for fine-tuning, we randomly sample 1500 source cases from each seed pool and generate follow-up cases with each MR. We construct MPs with follow-up cases that incur ICHs, compute their HD, and select the top-100 ICH-sensitive cases (total 600 cases) to fine-tune the testing subjects in RQ1∼RQ3. The performances of the fine-tuned models are evaluated on the set of follow-up cases generated in RQ1. For each dataset, we statistic the ICHs of all MRs and measure the models' performance with the ratio of ICHs.

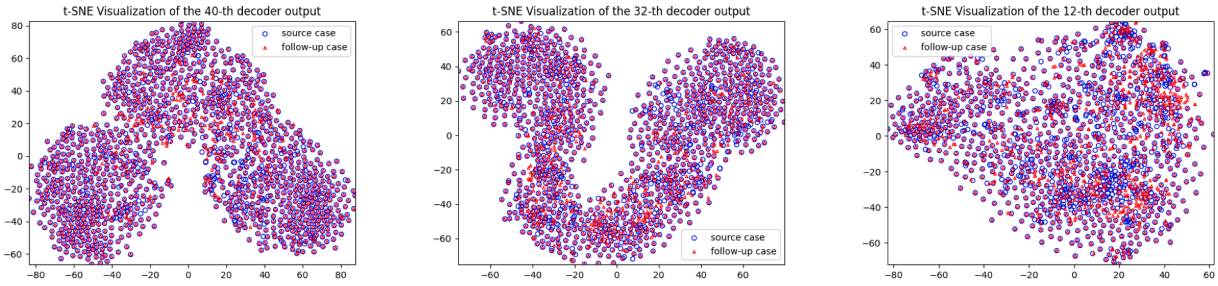
Table 6 shows the performances of the fine-tuned models. From the table, after fine-tuning, the ratio of detected ICHs significantly decreased

Table 3
Task instructions used to fine-tune target LLMs.

Task	Task instruction
NLI	Assess the connection between the following sentences and classify it as “entailment”, “neutral”, or “contradiction”:
DSD	Can these two statements be considered equal in meaning? Answer with “equivalent” or “not_equivalent”:
RC	Discover the best answer of the question based on the context. If the context doesn’t include an answer, respond with “unanswerable”:

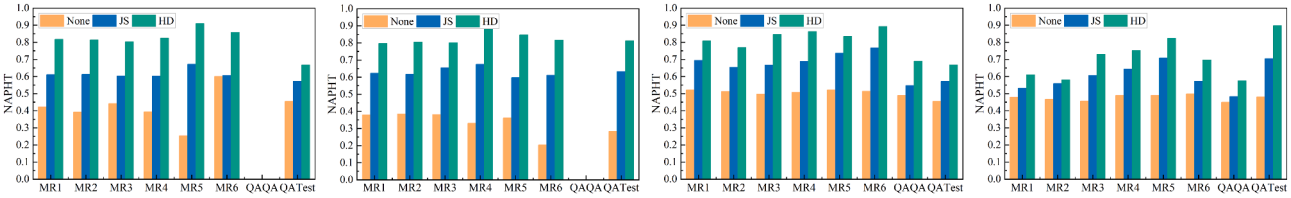
Table 4
Comparison of HTR for MRs between HalluTrigger and state-of-the-Art baselines across target models and datasets.

Target Model	Dataset	Attack Method							
		QAQA	QATest	MR1	MR2	MR3	MR4	MR5	MR6
Vicuna-13B	MNLI	–	8.72%	12.15%	12.11%	11.09%	14.58%	38.58%	52.36%
	QQP	–	9.68%	9.65%	9.27%	9.17%	17.89%	34.00%	82.69%
	SQuAD2	12.58%	16.97%	15.43%	15.59%	17.95%	24.41%	56.51%	53.61%
	NarrativeQA	19.28%	17.79%	19.33%	19.88%	21.25%	26.84%	62.10%	64.72%
LLaMA-3-8B	MNLI	–	10.30%	14.09%	13.59%	14.98%	19.40%	40.78%	57.13%
	QQP	–	11.80%	13.16%	9.81%	14.39%	18.10%	39.88%	82.90%
	SQuAD2	17.13%	22.09%	20.94%	20.69%	22.95%	30.63%	54.68%	52.24%
	NarrativeQA	19.93%	26.15%	28.18%	26.87%	27.22%	32.53%	66.20%	65.45%
BART-large	MNLI	–	18.91%	17.22%	18.85%	19.01%	18.68%	38.67%	57.13%
	QQP	–	24.09%	24.66%	23.44%	24.91%	24.79%	56.66%	91.12%
	SQuAD2	18.25%	25.85%	30.05%	34.66%	32.59%	33.55%	59.13%	57.31%
	NarrativeQA	–	–	–	–	–	–	–	–



(a) result on Vicuna-13B (b) result on LLaMA-3-8B (c) result BART-large

Fig. 4. Distribution of source cases (○) and their corresponding follow-up cases (△) in the representation space constructed from the outputs of the last decoder block of the target LLM.



(a) Result on MNLI (b) Result on QQP (c) Result on SQuAD2 (d) Result on NarrativeQA

Fig. 5. Prioritization performance of diversity metrics on Vicuna-13B: HD ranking outperforms, none worst.

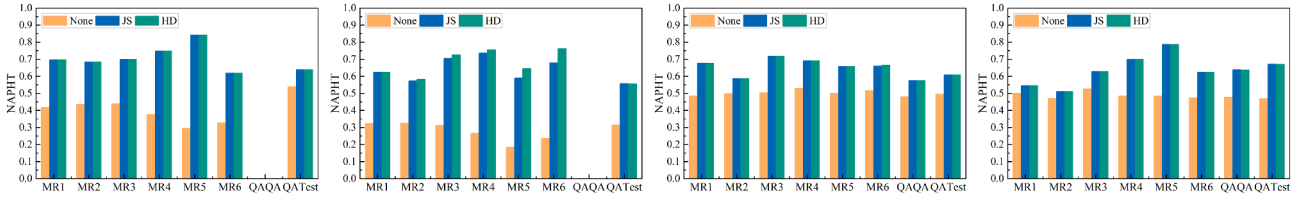
Table 5
TPR across target models and datasets.

Target Model	MNLI	QQP	SQuAD2	NarrativeQA
Vicuna-13B	99%	99%	98%	97%
LLaMA-3-8B	98%	99%	97%	97%
BART-large	97%	98%	97%	–

on each model and dataset. Specifically, the adversarial fine-tuning method performed best on the BART model with the QQP dataset (decreasing by 41.54%), and worst on the LLaMA model with the SQuAD2 dataset (decreasing by 23.98%).

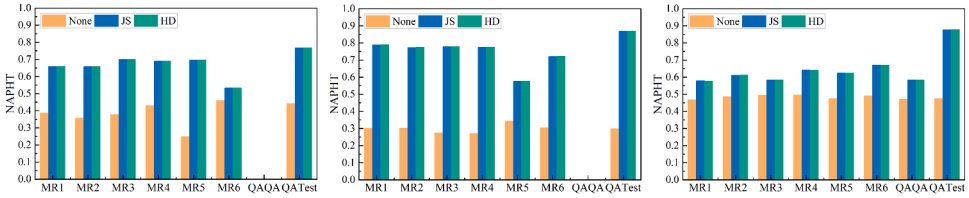
4.9. RQ4: ablation study

In this study, we aim to investigate the impact of the selection of the compute unit DB_k, which is directly reflected through changes in NAPHT. Figs. 9–11 show the results on BART-large, Vicuna-13B, and LLaMA-3-8B, respectively. From these figures, the value of NAPHT increases with the increment of k in most scenarios. Thus we can summarize that the efficiency of MP prioritization is improved as k increases, and a larger k tends to achieve a better performance in prioritizing the ICH-sensitive MPs in Γ'. In particular, the NAPHT of these MRs approach maximum when DB_k is set to a middle position. Therefore, we can set the intermediate position in target models as the compute unit to save diversity computing time while achieving near-optimal prioritization performance.



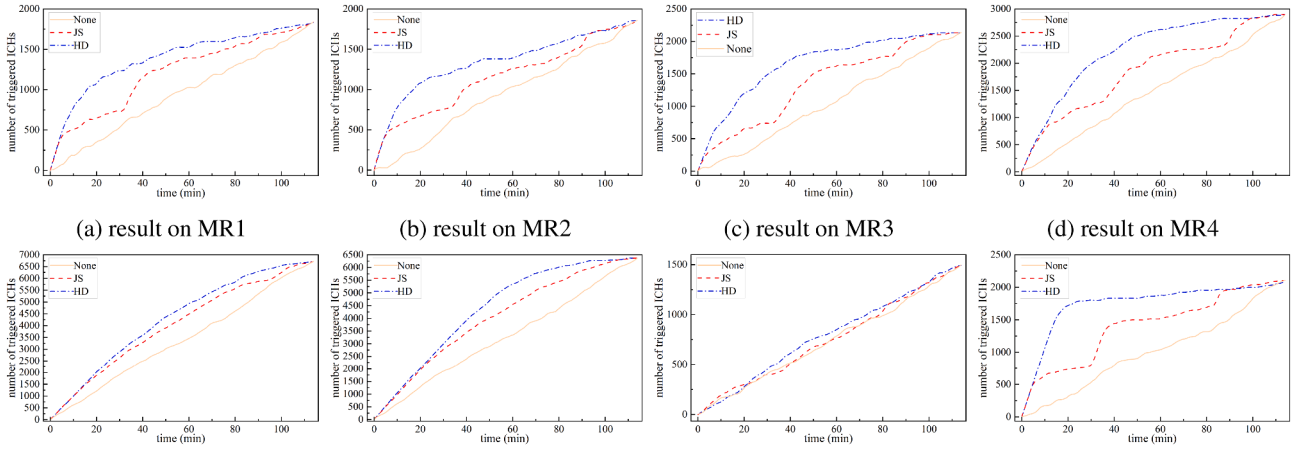
(a) Result on MNLI (b) Result on QQP (c) Result on SQuAD2 (d) Result on NarrativeQA

Fig. 6. Prioritization performance of diversity metrics on LLaMA-3-8B: HD most effective or comparable to JS, none worst.



(a) Result on MNLI (b) Result on QQP (c) Result on SQuAD2

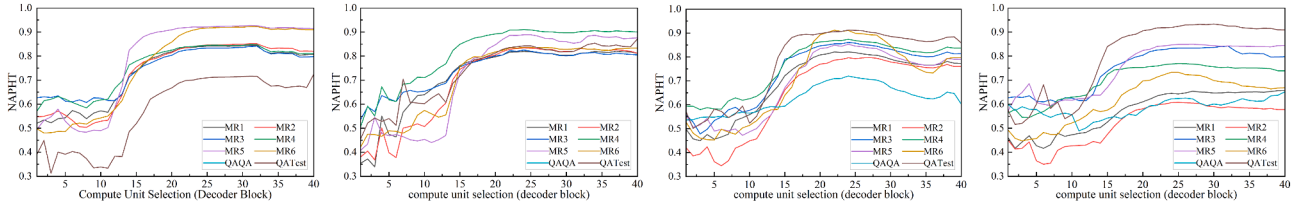
Fig. 7. Prioritization performance of diversity metrics on BART-large: HD most effective or comparable to JS, none worst.



(a) result on MR1 (b) result on MR2 (c) result on MR3 (d) result on MR4

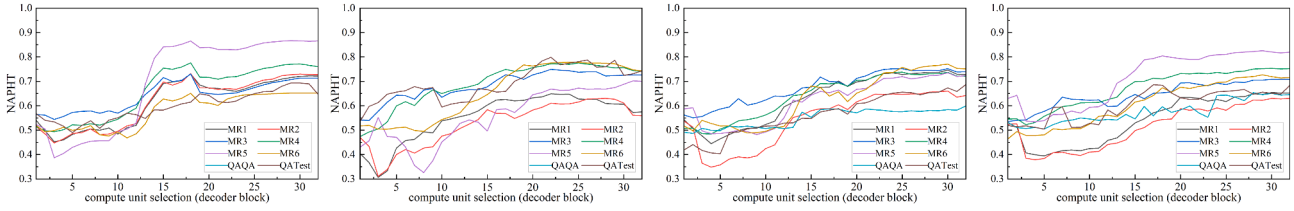
(e) result on MR5 (f) result on MR6 (g) result on QAQA (h) result on QATest

Fig. 8. The number of triggered ICHs over time for the unsorted and sorted MP sequences on SQuAD2 and Vicuna-13B.



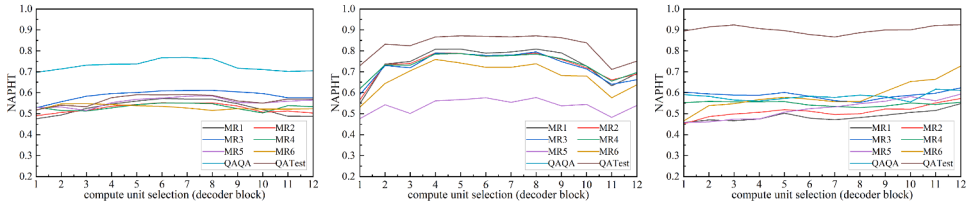
(a) Result on MNLI (b) Result on QQP (c) Result on SQuAD2 (d) Result on NarrativeQA

Fig. 9. Performance of HD-based MP prioritization with different decoder block as the compute unit on Vicuna-13B.



(a) result on MNLI (b) result on QQP (c) result on SQuAD2 (d) result on NarrativeQA

Fig. 10. Performance of HD-based MP prioritization with different decoder block as the compute unit on LLaMA-3-8B.



(a) result on MNLI (b) result on QQP (c) result on SQuAD2

Fig. 11. Performance of HD-based MP prioritization with different decoder block as the compute unit on BART-large.

Table 6
The ratio of reported input-conflicting hallucinations (ICHs) before and after adversarial fine-tuning.

Target Model		MNLI	QQP	SQuAD2	NarrativeQA
Vicuna-13B	before	52.36%	82.69%	56.51%	62.10%
	after	32.43%	55.01%	42.31%	44.67%
	decrease	↓38.06%	↓33.47%	↓25.13%	↓28.07%
LLaMA-3-8B	before	57.13%	82.90%	54.68%	66.20%
	after	35.91%	52.50%	41.57%	48.04%
	decrease	↓37.14%	↓36.67%	↓23.98%	↓27.43%
BART-large	before	57.13%	91.12%	59.13%	–
	after	34.72%	53.27%	39.87%	–
	decrease	↓39.22%	↓41.54%	↓32.57%	–

Table 7
Results of cross-model transferability rate (CTR), measuring the proportion of ICH-inducing follow-up cases transferred from the source model to the target model. Note that MR_{Char} is the average of the results of MR1-4.

(a)Vicuna-13B as source						
Dataset	LLaMA-3-8B			BART-large		
	MR_{Char}	MR5	MR6	MR_{Char}	MR5	MR6
MNLI	41.61%	61.87%	63.86%	47.45%	56.80%	73.80%
QQP	52.44%	65.40%	59.66%	47.56%	53.98%	42.72%
SQuAD2	47.20%	73.91%	85.34%	60.80%	74.13%	84.33%
NarrativeQA	41.50%	64.65%	55.39%	–	–	–
(b)LLaMA-3-8B as source						
Dataset	Vicuna-13B			BART-large		
	MR_{Char}	MR5	MR6	MR_{Char}	MR5	MR6
MNLI	44.83%	62.54%	80.67%	41.01%	57.09%	81.23%
QQP	40.15%	44.38%	63.16%	45.64%	45.56%	42.33%
SQuAD2	48.61%	73.91%	92.20%	62.34%	71.76%	93.31%
NarrativeQA	47.16%	59.46%	58.92%	–	–	–
(c)BART-large as source						
Dataset	Vicuna-13B			LLaMA-3-8B		
	MR_{Char}	MR5	MR6	MR_{Char}	MR5	MR6
MNLI	45.47%	61.50%	55.88%	48.69%	61.05%	65.66%
QQP	45.25%	48.51%	57.07%	56.10%	65.40%	57.39%
SQuAD2	53.91%	73.91%	93.04%	48.80%	72.61%	86.76%
NarrativeQA	–	–	–	–	–	–

5. Discussion

5.1. Threats to validity

Regarding internal validity, the primary threats stem from (1) the effectiveness of the proposed metamorphic relations (MRs) and efficiency metric, and (2) the configuration sensitivity of our method and the adopted diversity metrics. The proposed MRs are constructed following well-established text mutation principles widely used in prior NLP robustness and adversarial testing studies (Gao et al., 2018; Garg & Ramakrishnan, 2020; Jin et al., 2019; Li et al., 2018; Morris et al., 2020), and can be instantiated using existing techniques. The effectiveness of the proposed NAPHD metric, which extends NAPVD and NAPFD from traditional software and DNN testing, has been quantitatively validated in RQ2. In addition, we further verified this by randomly sampling 100 test cases under different diversity metrics and manually inspecting the results. Moreover, the MP prioritization strategy was evaluated across different configurations, diversity metrics, and target LLMs, showing stable and consistent performance, which mitigates concerns about configuration sensitivity.

Regarding external validity, we explicitly discuss two main aspects: (1) the representativeness of the selected tasks, datasets, and target models, and (2) the true positive rate of the reported ICHs. To address the former, we evaluated HalluTrigger on three representative NLP tasks (NLI, DSD, and RC), four widely used benchmark datasets (MNLI, QQP, SQuAD2, and NarrativeQA), and three state-of-the-art LLMs (Vicuna-13B, LLaMA-3-8B, and BART-large), all of which are commonly adopted in recent NLP research. To address the latter, we conducted a manual verification study by randomly sampling 100 reported ICHs per dataset; two authors independently checked whether these cases correspond to true positives. The resulting average true positive rates are 99.23%, 99.07%, and 97.33% for Vicuna-13B, LLaMA-3-8B, and BART-large, respectively, indicating that false positives are rare in practice.

5.2. Rationale of semantic-level MR

At the semantic level, we include only one MR based on semantic negation, as it provides a clear and deterministic inversion that enables reliable oracle-free detection of ICHs. In input-grounded tasks such as NLI, DSD, and RC, negation establishes an unambiguous expectation of output change defined solely by the input semantics and task specification. For example, in MNLI, negating the hypothesis should deterministically flip the label from “entailment” to “contradiction”, and failure to do so directly indicates an ICH. Other semantic transformations, such as paraphrasing or antonym substitution, often introduce ambiguous shifts that may preserve meaning or alter intent, making evaluation unreliable and potentially conflating ICHs with reasoning errors. Therefore, we restrict semantic-level MRs to negation to ensure precise violation attribution and robust ICH detection.

5.3. Cross-model transferability analysis

To evaluate of cross-model transferability of HalluTrigger, for each MR, we first collect the follow-up cases that successfully trigger ICHs on a source model. These ICH-inducing follow-up cases are then reused as fixed test inputs on other target models, under the same ICH detection criteria, without any additional optimization or model-specific adaptation. We define cross-model transferability rate (CTR), the percentage of the ICH-inducing cases that also trigger ICHs on a target model, to precisely measure the cross-model transferability of our method. Given a source model $\mathcal{M}$ and a target model $\mathcal{M}'$, let $N^{ICH}_{\mathcal{M}}$ and $N^{ICH}_{\mathcal{M}'}$ denote the numbers of ICHs triggered under a metamorphic relation MR on $\mathcal{M}$ and $\mathcal{M}'$, respectively. The CTR is then defined as follows:

$$CTR_{S,\mathcal{M},\mathcal{M}',MR} = \frac{N^{ICH}_{\mathcal{M}'}}{N^{ICH}_{\mathcal{M}}} \tag{18}$$

Table 7 show that the CTR exceed 40.15% across all levels. Referring to prior work on adversarial text attacks and robustness testing (Naseer et al., 2019; Wu & Zhu, 2020; Yuan et al., 2021), such substantial cross-model transferability indicates that the triggered ICHs are not isolated or model-specific failures, but instead reflect systematic and reproducible vulnerabilities shared by different LLM architectures.

5.4. False positive case study

Furthermore, in order to enhance the comprehensiveness of our method, we provide a MNLI failure case example as follows. Consider the following source and follow-up input pair under MR5 (SimilarWordSubstitution):

- *(source case) Premise: They're sloppy, imperfect products, trying desperately to live up to an ideal of progress. Hypothesis: These products are imperfect.*
- *(follow-up case) Premise: They're sloppy, imperfect products, trying desperately to live up to an ideal of progress. Hypothesis: These products are terrible.*

Although this substitution introduces a stronger evaluative judgment that should shift the label from "entailment" to "neutral," the target LLM predicts "entailment" for both inputs, leading HalluTrigger to flag a false positive caused by the model's conflation of descriptive imperfection with negative evaluation.

6. Conclusion

In this paper, we introduce HalluTrigger, a grey-box method to trigger the ICHs in the popularly used open-source LLMs. It attacks the target models at the character, sentence, and semantic levels with six semantic-guided MRs. To boost the revealing of ICHs, we use the hidden outputs in target LLMs to identify the ICH-sensitive MPs and make the first attempt to prioritize the generated MPs with two diversity metrics. Besides, we propose a new metric to measure the performance of the MP prioritization method. Through a comprehensive evaluation, HalluTrigger outperforms the SOTA baselines in terms of attack effectiveness, attack quality, and generalizability. The MP prioritization method can stably improve the attack efficiency both for HalluTrigger and the SOTA baselines.

CRediT authorship contribution statement

Chongyang Liu: Conceptualization, Methodology, Data curation, Software, Investigation, Writing – original draft; **Xiaoning Ren:** Validation, Data curation, Visualization, Writing – review & editing; **Yinxing Xue:** Conceptualization, Methodology, Supervision, Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research is supported in part by the National Nature Science Foundation of China (grant No. 61972373).

References

Alzantot, M., Sharma, Y., Elgohary, A., Ho, B.-J., Srivastava, M., & Chang, K.-W. (2018). Generating natural language adversarial examples. arXiv:1804.07998, .

Anthropic (2023). Introducing claude. https://www.anthropic.com/index/introducing-claude.

Bang, Y., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., Lovenia, H., Ji, Z., Yu, T., Chung, W. et al. (2023). A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. arXiv:2302.04023.

Berend, D., Xie, X., Ma, L., Zhou, L., Liu, Y., Xu, C., & Zhao, J. (2020). Cats are not fish: Deep learning testing calls for out-of-distribution awareness. In *Proceedings of the 35th IEEE/ACM International conference on automated software engineering* (pp. 1041–1052).

Bird, S., Loper, E., & Klein, E. (2001). NLTK. https://www.nltk.org/.

Cao, Y., Zhou, Z. Q., & Chen, T. Y. (2013). On the correlation between the effectiveness of metamorphic relations and dissimilarities of test case executions. In *2013 13th International conference on quality software* (pp. 153–162). IEEE.

Chen, S., Jin, S., & Xie, X. (2021). Testing your question answering software via asking recursively. In *2021 36th IEEE/ACM International conference on automated software engineering (ASE)* (pp. 104–116). IEEE.

Chen, T. Y., Cheung, S. C., & Yiu, S. M. (2020). Metamorphic testing: A new approach for generating next test cases. arXiv:2002.12543, .

Chen, T. Y., Huang, D. H., Tse, T. H., & Zhou, Z. Q. (2004). Case studies on the selection of useful relations in metamorphic testing. In *Proceedings of the 4th Ibero-American symposium on software engineering and knowledge engineering (JIISIC 2004)* (pp. 569–583). Citeseer.

Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E. et al. (2023). Vicuna: An open-source chatbot impressing GPT-4 with 90%* chatgpt quality. See https://vicuna.lmsys.org (accessed 14 April 2023), *2* (3), 6.

Dale, D., Voita, E., Barrault, L., & Costa-jussà, M. R. (2022). Detecting and mitigating hallucinations in machine translation: Model internal workings alone do well, sentence similarity even better. arXiv:2212.08597, .

Duan, H., Dziedzic, A., Papernot, N., & Boenisch, F. (2024). Flocks of stochastic parrots: Differentially private prompt learning for large language models. In *Advances in neural information processing systems*, *vol.36*.

Dziri, N., Rashkin, H., Linzen, T., & Reitter, D. (2022). Evaluating attribution in dialogue systems: The BEGIN benchmark. *Transactions of the Association for Computational Linguistics*, *10*, 1066–1083.

Gao, J., Lanchantin, J., Soffa, M. L., & Qi, Y. (2018). Black-box generation of adversarial text sequences to evade deep learning classifiers. In *2018 IEEE Security and privacy workshops (SPW)* (pp. 50–56). IEEE.

Gao, T., Yao, X., & Chen, D. (2021). Simcse: Simple contrastive learning of sentence embeddings. arXiv:2104.08821, .

Garg, S., & Ramakrishnan, G. (2020). Bae: BERT-based adversarial examples for text classification. arXiv:2004.01970, .

Guerreiro, N. M., Voita, E., & Martins, A. F. T. (2022). Looking for a needle in a haystack: A comprehensive study of hallucinations in neural machine translation. arXiv:2208.05309, .

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2020). Measuring massive multitask language understanding. arXiv:2009.03300, .

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). Lora: Low-rank adaptation of large language models. arXiv:2106.09685, .

Hu, M., Yang, Y., Pan, D., Guo, Z., Xiao, L., Lin, D., & Zhao, S. (2025). Syntactic paraphrase-based synthetic data generation for backdoor attacks against chinese language models. *Information Fusion*, 103376.

Huang, J.-t., Zhang, J., Wang, W., He, P., Su, Y., & Lyu, M. R. (2022). Aeon: A method for automatic evaluation of nlp test cases. In *Proceedings of the 31st ACM SIGSOFT International symposium on software testing and analysis* (pp. 202–214).

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B. et al. (2023). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv:2311.05232, .

Jin, D., Jin, Z., Zhou, J. T., & Szolovits, P. (2019). Is BERT really robust? Natural language attack on text classification and entailment. arXiv:1907.11932, *2*, 10.

Khashabi, D., Min, S., Khot, T., Sabharwal, A., Tafjord, O., Clark, P., & Hajishirzi, H. (2020). Unifiedqa: Crossing format boundaries with a single qa system. arXiv:2005.00700, .

Kočiskỳ, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K. M., Melis, G., & Grefenstette, E. (2018). The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, *6*, 317–328.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2020). Albert: A lite BERT for self-supervised learning of language representations. In *International conference on learning representations*.

Lee, N., Ping, W., Xu, P., Patwary, M., Fung, P. N., Shoeybi, M., & Catanzaro, B. (2022). Factuality enhanced language models for open-ended text generation. *Advances in Neural Information Processing Systems*, *35*, 34586–34599.

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2019). Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. arXiv:1910.13461, .

Li, J., Ji, S., Du, T., Li, B., & Wang, T. (2018). Textbugger: Generating adversarial text against real-world applications. arXiv:1812.05271, .

Li, N., Li, Y., Liu, Y., Shi, L., Wang, K., & Wang, H. (2024). Halluvault: A novel logic programming-aided metamorphic testing framework for detecting fact-conflicting hallucinations in large language models. arXiv:2405.00648, .

Lin, S., Hilton, J., & Evans, O. (2021). Truthfulqa: Measuring how models mimic human falsehoods. arXiv:2109.07958, .

Liu, Z., Feng, Y., Yin, Y., Sun, J., Chen, Z., & Xu, B. (2022). Qatest: A uniform fuzzing framework for question answering systems. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering* (pp. 1–12).

LlamA, M. (2024). LLaMA 3. https://llama.meta.com/llama3/.

Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, *9* (11), 2579–2605.

McCoy, R. T., Pavlick, E., & Linzen, T. (2019). Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. arXiv:1902.01007, .

Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-t., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. arXiv:2305.14251, .

Morris, J. X., Lifland, E., Yoo, J. Y., Grigsby, J., Jin, D., & Qi, Y. (2020). Textattack: A framework for adversarial attacks, data augmentation, and adversarial training in nlp. arXiv:2005.05909, .

Naseer, M. M., Khan, S. H., Khan, M. H., Shahbaz Khan, F., & Porikli, F. (2019). Cross-domain transferability of adversarial perturbations. In *Advances in neural information processing systems*, *vol. 32*.

OpenAI (2023). Gpt-4 technical report.

Rajpurkar, P., Jia, R., & Liang, P. (2018). Know what you don't know: Unanswerable questions for SQuAD. arXiv:1806.03822, .

Rawlinson, G. (2007). The significance of letter position in word recognition. *IEEE Aerospace and Electronic Systems Magazine*, *22* (1), 26–27.

Rothermel, G., Untch, R. H., Chu, C., & Harrold, M. J. (1999). Test case prioritization: An empirical study. In *Proceedings IEEE international conference on software maintenance-1999 (ICSM'99).'software maintenance for business change'(cat. no. 99CB36360)* (pp. 179–188). IEEE.

Shen, Q., Chen, J., Zhang, J. M., Wang, H., Liu, S., & Tian, M. (2022). Natural test generation for precise testing of question answering software. In *Proceedings of the 37th IEEE/ACM International conference on automated software engineering* (pp. 1–12).

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2018). Glue: A multi-task benchmark and analysis platform for natural language understanding. arXiv:1804.07461, .

Wang, S., Thompson, L., & Iyyer, M. (2021). Phrase-BERT: Improved phrase embeddings from bert with an application to corpus exploration. arXiv:2109.06304, .

Williams, A., Nangia, N., & Bowman, S. R. (2017). A broad-coverage challenge corpus for sentence understanding through inference. arXiv:1704.05426, .

Wu, L., & Zhu, Z. (2020). Towards understanding and improving the transferability of adversarial examples in deep neural networks. In *Asian conference on machine learning* (pp. 837–850). PMLR.

Xie, X., Yin, P., & Chen, S. (2022). Boosting the revealing of detected violations in deep learning testing: A diversity-guided method. In *Proceedings of the 37th IEEE/ACM International conference on automated software engineering* (pp. 1–13).

Yang, L., Wei, C., Yang, J., Xia, W., Yang, Y., Luo, Y., Niyato, D., Sun, L., & Liu, Z. (2025). Fuzzcode: Code large language model-based fuzz testing for industrial iot programs. *IEEE Internet of Things Journal*.

Yu, J., Wang, X., Tu, S., Cao, S., Zhang-Li, D., Lv, X., Peng, H., Yao, Z., Zhang, X., Li, H. et al. (2023). Kola: Carefully benchmarking world knowledge of large language models. *International Conference on Learning Representations*, . *2024*, 44594–44637.

Yuan, L., Chen, Y., Cui, G., Gao, H., Zou, F., Cheng, X., Ji, H., Liu, Z., & Sun, M. (2024). Revisiting out-of-distribution robustness in NLP: Benchmarks, analysis, and LLMs evaluations. In *Advances in neural information processing systems*, *vol. 36*.

Yuan, L., Zheng, X., Zhou, Y., Hsieh, C.-J., & Chang, K.-W. (2021). On the transferability of adversarial attacks against neural text classifier. In *Proceedings of the 2021 Conference on empirical methods in natural language processing* (pp. 1612–1625).

Zha, Y., Yang, Y., Li, R., & Hu, Z. (2023). Alignscore: Evaluating factual consistency with a unified alignment function. arXiv:2305.16739, .

Zhang, J., Chi, J., Li, Z., Cai, K., Zhang, Y., & Tian, Y. (2024a). Badmerging: Backdoor attacks against model merging. In *Proceedings of the 2024 on ACM SIGSAC Conference on computer and communications security* (pp. 4450–4464).

Zhang, T., Ladhak, F., Durmus, E., Liang, P., McKeown, K., & Hashimoto, T. B. (2024b). Benchmarking large language models for news summarization. *Transactions of the Association for Computational Linguistics*, *12*, 39–57.

Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y. et al. (2023). Siren's song in the AI ocean: a survey on hallucination in large language models. arXiv:2309.01219, .

Zhao, S., Xu, X., Xiao, L., Wen, J., & Tuan, L. A. (2025). Clean-label backdoor attack and defense: An examination of language model vulnerability. *Expert Systems with Applications*, *265*, 125856.

Zhong, M., Yin, D., Yu, T., Zaidi, A., Mutuma, M., Jha, R., Awadallah, A. H., Celikyilmaz, A., Liu, Y., Qiu, X. et al. (2021). Qmsum: A new benchmark for query-based multi-domain meeting summarization. arXiv:2104.05938, .

Zhu, K., Wang, J., Zhou, J., Wang, Z., Chen, H., Wang, Y., Yang, L., Ye, W., Gong, N. Z., Zhang, Y. et al. (2023). Promptbench: Towards evaluating the robustness of large language models on adversarial prompts. arXiv:2306.04528, .